Who Is Responsible When AI Agents Disagree? Causal Evidence That Cross-Agent Memory Contradictions Cause 73-Percentage-Point Task Failures

Anonymous Author(s)
Anonymous Institution

ABSTRACT

AI agents increasingly share persistent memory stores to collaborate. When two agents write contradictory facts about the same entity—one writes "API key X is available," another writes "API key X returned 403"—downstream agents receive conflicting evidence. Despite the growing deployment of multi-agent systems, no prior work has quantified the causal impact of these contradictions on task performance. We present the first causal quantification: in a controlled experiment (n=100, McNemar $p < 0.0001$), agents receiving stale context produce 39 percentage points more errors than agents receiving correct context. We further identify a critical asymmetry: *cross-agent* contradictions cause a 73-percentage-point accuracy loss (9% vs. 82%, n=11), while *self-corrections* have negligible impact (0.7pp, n=300). To address this, we implement a bi-temporal memory system with hash-chained audit trails (MemTether), prove that overwrite-style stores cannot support temporal queries without $\Omega(k)$ replay, and achieve 100% temporal accuracy on 50 real correction events (vs. 14% for overwrite-style stores). We evaluate our contradiction detection method on ConflictBank's temporal subset (n=2,000), achieving $F_1 = 0.903$ with perfect precision, and demonstrate a 53.6pp retrieval improvement through contradiction-aware filtering. We release the system, dataset, and all evaluation scripts as open source.

ACM Reference Format:
Anonymous Author(s). 2026. Who Is Responsible When AI Agents Disagree? Causal Evidence That Cross-Agent Memory Contradictions Cause 73-Percentage-Point Task Failures. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

AI agents are deployed in multi-agent settings where they share persistent memory: code assistants that remember project conventions, research agents that build on prior findings, customer-service agents that share knowledge about products and policies. In these deployments, agents independently write facts to a shared store. When two agents reach different conclusions about the same entity—one writes "the deployment is healthy," another writes "the deployment crashed"—the memory contains a *cross-agent contradiction.*

Despite the rapid growth of agent memory systems (mem0 [1], Zep [2], Letta [3]), no prior work has asked: *what happens to downstream task performance when agents disagree?*

We answer this question with a causal experiment. In a controlled setting (n=100), agents receiving stale (outdated) memory context produce 39 percentage points more errors than agents receiving correct context (McNemar $p < 0.0001$). We further discover a critical asymmetry: *cross-agent* contradictions—where two different agents wrote conflicting facts—cause a 73pp accuracy loss, while *self-corrections*—where an agent corrects its own earlier statement—cause only 0.7pp.

This asymmetry matters for accountability. When an AI system makes a harmful decision based on contradictory memory, who is responsible? The agent that wrote the outdated fact? The agent that acted on it? The system that failed to detect the contradiction? Current memory systems provide no mechanism to answer these questions: they lack audit trails, provenance tracking, and evidence preservation.

We present MemTether, a local-first memory governance system that addresses these accountability gaps through three mechanisms: (1) *bi-temporal supersession chains* that track when facts were corrected and by whom, (2) *hash-chained audit trails* that detect any tampering with the audit log, and (3) *contradiction detection* that identifies and resolves conflicting facts before they reach downstream agents.

2 BACKGROUND AND RELATED WORK

2.1 Agent Memory Systems

mem0 [1] provides hosted memory with LLM-based fact extraction and vector retrieval. When `update()` is called, the old embedding is replaced in-place. LangMem [4] integrates memory into LangGraph. Letta [3] (formerly MemGPT) manages hierarchical memory with a virtual context. These systems are designed for single-user deployments: each user has a separate memory space, and cross-agent contradictions cannot arise.

Caura (formerly MemClaw [5]) is the most directly related system: a production multi-tenant, multi-agent memory service with governed shared memory. Caura implements tenant isolation, agent trust tiers, audit logging, contradiction detection (via RDF triple comparison and LLM semantic analysis), and supersession. It is deployed in production at eToro. However, Caura does not implement hash-chained audit trails (its audit log is a standard database log, not a cryptographic chain), does not provide a formal bi-temporal model, does not prove impossibility results, and has not conducted causal experiments measuring downstream task impact.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2.2 Memory Governance

AECG [6] defines "memory pollution" (outdated or weakly supported procedures persisting in memory) and "scope collapse" (knowledge escaping its coordination scope), proposing a governance framework for multi-agent memory. However, AECG is a conceptual framework without system implementation, production data, or causal experiments.

MutMem [7] implements cryptographically authorized mutation of persistent agent memory, binding each weight change to signed evidence with SHA-256 commitments. This is complementary to our hash-chained audit: MutMem secures *mutation authorization* while we secure *audit trail integrity*.

Beyond Provenance [8] develops an economic model of AI memory governance, showing that provenance alone does not settle rights and responsibilities. This theoretical work motivates our practical system: MemTether provides the infrastructure that Beyond Provenance argues is necessary.

Revoked but Still Authoritative [9] empirically demonstrates that no agent memory system enforces revocation by default: revoked facts continue to be returned wherever the revocation label is visible to the retrieval layer. This finding resonates with our 73pp result: both show that memory systems fail silently when facts change.

Lineage-Aware Memory Governance [10] addresses enterprise memory governance through derivation-gated access control, focusing on privacy leakage and KPI consistency. When to Remember [11] studies retention decisions conditioned on semantic category. Epistemics of Agent Memory [12] studies consolidation decisions in long-horizon agents.

2.3 Memory Security

MemMark [13] proposes state-evolution attribution watermarking for agent memory. Securing LLM-Agent Memory [14] studies memory poisoning attacks and defenses. These works address malicious actors; our work addresses contradictions that arise from *legitimate* multi-agent disagreement.

2.4 Positioning

Table 1 summarizes the competitive landscape. No prior system combines bi-temporal versioning, hash-chained audit, impossibility results, and causal experiments measuring downstream task impact.

Table 1: Competitive landscape.

System	Bi-temporal	Hash audit	Causal exp.	Contradict. detect
mem0	No	No	No	No
Zep	Yes	No	No	No
Caura	No	partial	No	Yes
AECG	No	No	No	conceptual
Ours	Yes	Yes	Yes	Yes

3 THE PROBLEM: CROSS-AGENT CONTRADICTIONS

3.1 Setup

We study a production deployment of MemTether: a local-first memory system where 13 AI agent clients (Codex CLI, WorkBuddy CN/Intl, OpenClaw, ZCode, Tabbit, and others) share a single SQLite database via file-level pointers. Over 26 days, these agents wrote 1,768 active facts, performed 348 supersession corrections, and generated 2,346 audit log entries.

3.2 Contradiction Discovery

We scan all 24,907 cross-source entity pairs using two methods:

Temporal-numeric overlap. Extract year sets Y_a, Y_b and number sets N_a, N_b from each fact. Detect conflict if $Y_a \neq Y_b$ or ($N_a \cap N_b \neq \emptyset \wedge N_a \triangle N_b \neq \emptyset$).

Entity-level polarity clash. Extract (entity, polarity) pairs from individual clauses using a governance lexicon. Detect conflict if the same entity has opposite polarity across sources.

The scan identifies 198 polarity-clash candidates. We classify these into four types using deterministic rules: genuine temporal flips (11, 5.6%), entity polarity false positives (154, 77.8%), and cases requiring human review (33, 16.7%).

3.3 External Validation on ConflictBank

To validate our detection method beyond our own production data, we evaluate on ConflictBank [15], a benchmark with 7.4M claim-evidence pairs. From the temporal conflict subset (1,000 positive + 1,000 negative pairs), our temporal-numeric method achieves:

Table 2: ConflictBank temporal conflict detection (n=2,000).

Method	Precision	Recall	F_1	Latency
Ours	1.000	0.814	**0.897**	0.133 ms
GLM-5.3-flash	1.000	0.060	0.113	1.56 s

Our method achieves perfect precision (zero false positives) with 11,704No speedup over LLM-based detection.

4 THE IMPOSSIBILITY RESULT

Definition 1. *An* overwrite-style store M *maintains a single current value per key;* `update(key, value)` *destroys the previous value.*

Definition 2. *An* as-of query *at time t returns the value that was current at t.*

Theorem 1. *For any overwrite-style store M and wrapper W around M: (1) W cannot answer as-of queries using M's interface alone; (2) W must maintain an auxiliary log and perform $\Omega(k)$ replay, where k is the number of updates; (3) if the auxiliary log is itself bi-temporal, W has replaced M.*

Proof. (1) M exposes only the latest value; historical values are inaccessible. (2) To find the value at time t, W must scan its log for the last entry before t: worst case $\Omega(k)$. (3) If the log supports

indexed temporal queries, it is a bi-temporal store—the wrapper is redundant. □

Corollary 1. *Temporal correctness requires bi-temporal storage at the storage layer.*

5 EVALUATION

5.1 Temporal Accuracy

We construct 50 questions from real production corrections. Each tests whether the system can retrieve both the old value ("What was true before?") and the new value ("What is true now?").

Table 3: Temporal accuracy (n=50 real corrections).

System	Past	Now	Temporal Acc.
Bi-temporal (ours)	50/50	50/50	**100%**
Overwrite (mem0)	7/50	50/50	14%

5.2 As-of Latency

Table 4: As-of query latency.

k corrections	Ours (ms)	Wrapper (ms)
1	0.167	25.5
500	0.245	27.0

Our system maintains flat $O(\log n)$ latency across 500× data scaling.

5.3 Causal Experiment: Stale Context Causes Task Failures

Design. Sample 100 supersession events. For each, construct two contexts: correct (new fact) and stale (old fact). Ask GLM-5.3-flash (temperature=0): "What is the current status of [entity]?" Grade against the gold (new) fact.

Table 5: Downstream accuracy (n=100).

Context	Correct	Wrong	Accuracy
Correct	99	1	99.0%
Stale	60	40	60.0%
McNemar $p < 0.0001$			39.0pp

5.4 The Cross-Agent Asymmetry

We separate the analysis by conflict type:

Finding 1. *Cross-agent contradictions cause a 73pp downstream accuracy loss. Self-corrections have negligible impact. The temporal gap is specifically a* cross-agent *problem: agents correcting their own errors maintain consistency, but contradictions between agents create irrecoverable confusion.*

Table 6: Downstream accuracy by conflict type.

Type	n	Correct ctx	Stale ctx	Loss
Cross-agent	11	82%	9%	**73pp**
Self-correction	300	100%	99.3%	0.7pp

5.5 Contradiction-Aware Retrieval

When retrieval returns contradictory results, our post-retrieval filter detects the contradiction (using entity-level polarity clash), removes the older fact, and returns the deduplicated results.

Table 7: Contradiction-aware retrieval A/B test.

Condition	Correct	Accuracy
Without filter	12/28	43%
With filter	27/28	**96%**

5.6 Counterfactual Analysis

We replay all supersession events in counterfactual mode (restore old facts to active) and re-scan for contradictions:

Table 8: Supersession prevents contradictions.

Dataset	Current	Without	Prevention
Production	11	23	52%
Synthetic	13	33	61%

6 ACCOUNTABILITY FRAMEWORK

6.1 The Accountability Gap

When an AI system produces a harmful output based on contradictory memory, current systems provide no mechanism to answer: Which fact was used? Who wrote it? When was it corrected? Was the correction ignored? Without audit trails, these questions are unanswerable.

6.2 Memory Court

MemTether's Memory Court provides a five-part case file for any fact:

(1) **Content**: The current value.
(2) **Temporal validity**: `valid_from` to `valid_to` (T-axis) and `recorded_at` to `invalidated_at` (T'-axis).
(3) **Provenance**: Who wrote each version, when, and why it was corrected.
(4) **Conflict record**: Which contradictions were detected and how they were resolved.
(5) **Integrity proof**: SHA-256 hash chain over all audit log entries, verifiable without trusted third parties.

6.3 Hash-Chained Audit Trail

Every memory operation (write, correct, retire, search) is logged. Every 10 entries, a SHA-256 anchor is computed: $\text{anchor}_i = \text{SHA256}(\text{anchor}_{i-1} \| \text{hash}(\text{entry}_{1..10}))$. Any modification to a logged entry breaks the chain and is detected by `verify_chain()`. The chain covers 2,346 audit entries across 348 correction events.

6.4 EU AI Act Compliance

EU AI Act Article 12 requires that high-risk AI systems maintain automatically generated event logs sufficient to reconstruct events. Our hash-chained audit log with bi-temporal supersession chains satisfies this requirement: every memory write, correction, and conflict resolution is logged with timestamps, agent identity, and content hashes, and the log's integrity is cryptographically verifiable.

7 LIMITATIONS

- n=11 for cross-agent contradictions: the 73pp estimate has wide confidence intervals.
- Single production deployment and single LLM judge (GLM-5.3-flash).
- Detection method is specific to temporal conflicts; semantic and misinformation conflicts require different approaches.
- Entity extraction requires pre-built `fact_entities` links.
- The production system is a single-machine SQLite prototype; scaling to distributed deployments requires additional engineering.

8 CONCLUSION

We presented the first causal quantification of cross-agent memory contradictions in multi-agent AI systems. Our findings: cross-agent contradictions cause a 73pp downstream accuracy loss while self-corrections have negligible impact. We proved that temporal correctness requires storage-layer support, implemented a bi-temporal system achieving 100% temporal accuracy with $O(\log n)$ latency, and evaluated our contradiction detection on ConflictBank (F_1 = 0.903). We released the system, dataset, and all evaluation scripts as open source.

REFERENCES

[1] P. Chhikara et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.
[2] R. Rasmussen et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956, 2025.
[3] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
[4] LangChain. LangMem: Memory Integration for LangGraph. Software, 2025.
[5] Caura (formerly MemClaw): Governed Shared Memory for Multi-Agent LLM Systems. arXiv:2606.24535, 2026. GitHub: github.com/caura-ai/caura
[6] AECG: Asymmetric Experience Consolidation and Governance in Multi-Agent Systems. arXiv:2610.05176, 2026.
[7] MutMem: Cryptographically Authorized Mutation in Persistent Agent Memory. arXiv:2608.02843, 2026.
[8] Beyond Provenance: The Economics and Governance of Personalized AI Memory. arXiv:2609.25521, 2026.
[9] Revoked but Still Authoritative: An Empirical Study of Revocation Enforcement in Agent-Memory Systems. arXiv:2609.08258, 2026.
[10] Lineage-Aware Memory Governance: A Derivation-Gated Framework. arXiv:2610.07258, 2026.
[11] When to Remember, When to Abstain: Category-Conditioned Retention. arXiv:2610.07100, 2026.
[12] The Epistemics of Agent Memory: Measuring, and Governing, the Consolidation Decision. arXiv:2609.33013, 2026.
[13] MemMark: State-Evolution Attribution Watermarking. arXiv:2605.25002, 2026.
[14] Sdhash [LM Agent Long-Term Memory Against Poisoning. arXiv:2606.24322, 2026.
[15] ConflictBank: A Benchmark for Evaluating Knowledge Conflicts. arXiv, 2024.
[16] Supersede: Memory Update Gap in LLM Agents. arXiv:2606.27472, 2026.
[17] C. Wu et al. LongMemEval: Benchmarking Long-Term Memory in LLM Agents. arXiv:2410.10813, 2024.
[18] Gamma: Grounded Multi-Agent Memory with Mechanical Audit. arXiv:2610.12793, 2026.